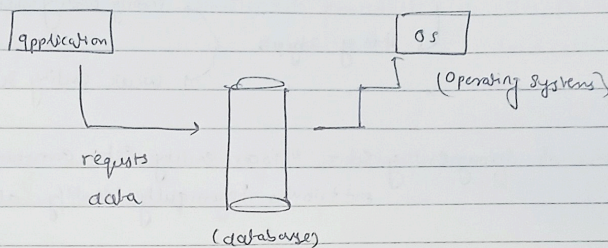


## ~ Database Management Systems ~

\* Database - Collection of related data representing some aspect of the real world → this data is stored such that interrelated remains together

\* DBMS (Database Management System) - Software for storing and retrieving user's data while considering appropriate security measures

↳ managed by group of programs → manipulating database (helps users and third party users)

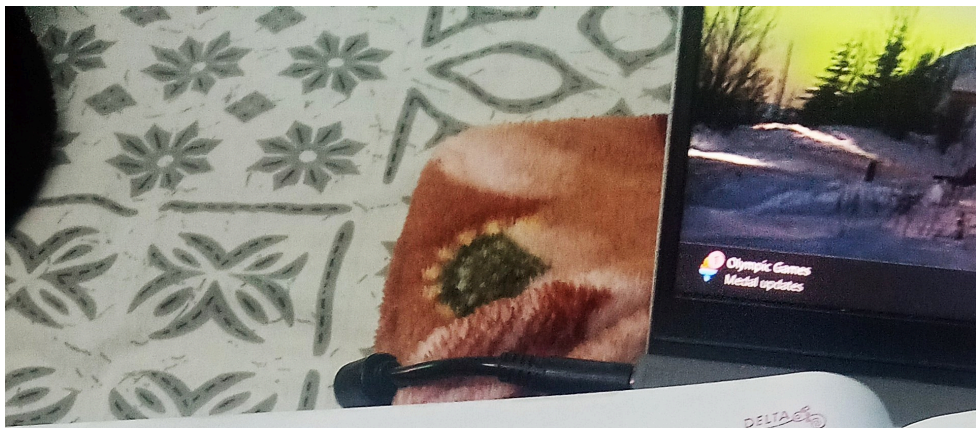


features of DBMS →

- less data redundancy and inconsistency
- difficulty in access data reduced
- provides atomicity, data isolation, integrity, helps concurrent access by multiple users
- Avoids security problems

\* ER diagram - (Entity - relationship diagram), a conceptual model giving graphical representation of logical structure of database

↳ Shows all relationships and constraints among different components



DELTA

Date / /

Pg No

→ ER diagram →

- Entity set
- attributes
- relationship set

Roll no

name

Age

Student

Roll no → primary key used to identify each entity uniquely  
(as many students can have same age or name)

Entity is used to represent any real world object

\* entity set - It is a set of same type of entities

↳ it is of 2 types

- strong entity set
- weak entity set

(i) strong entity set - It is an entity set containing sufficient attributes to uniquely identify all its entities

↓

having primary key

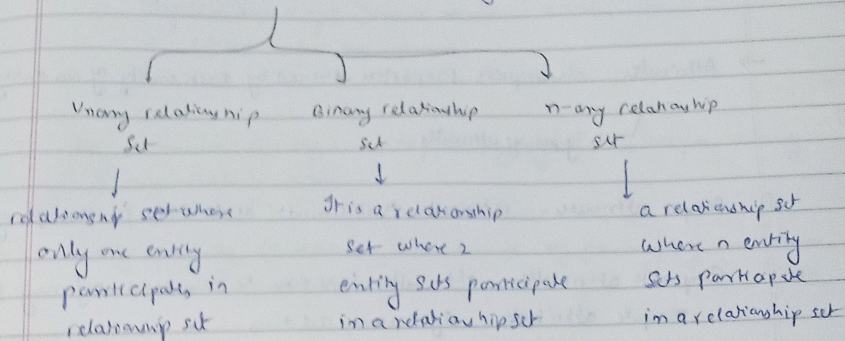
(ii) weak entity set - doesn't contain sufficient attributes to uniquely identify its ~~set~~ entities

↳ not having any primary key but may have a partial key (discriminator)

Can identify a group of entities from entity set

## \* Relationship:-

It is defined as an association among several entities



\* **Cardinality constraint** - It defines maximum no. of relationship instances in which entity can participate

• **One to one** → an entity in set A can be associated with at most 1 entity in set B  
(set B → at most 1 entity in set A)

• **One to many** → entity in set A can be associated with any no. of entities in set B  
(entity in set B → at most 1 entity in set A)

• **many to one** → an entity in set A can be associated with at most one entity in set B  
(set B entities → to any no. of entities in set A)

many to many  $\rightarrow$  entity in set A can be associated with any no. of entities in set B and an entity in set B can be associated with any number of entities in set A

$\rightarrow$  Attributes - descriptive properties owned by each entity of an entity set

$\rightarrow$  Types of attributes -

i) Simple attributes - attributes that cannot be divided further for eg. Age

(ii) Composite attributes - attributes that can be composed of many other simple attributes eg - name, address

(iii) multivalued attributes - attributes that can take more than one value for a given entity in entity set  $\rightarrow$  mobile no, email id etc.

↳ derived attributes - these attributes can be derived from other attributes (s) eg - Age  $\rightarrow$  derived from DOB

iv) Key attribute - used to identify an entity uniquely in an entity set  
eg - Roll no

Constraints

There are the restrictions imposed on the database contents and operations  $\rightarrow$  ensures correctness of data in database

i) Domain Constraint - It defines domains are a set of values for an attribute

$\rightarrow$  specifies that the value taken by attribute must be atomic value from its domain

### Tuple

- (ii) ~~For~~ Uniqueness constraint - It specifies that all the tuples must be necessarily unique in any relation

(tuple - a single row or record in a table  
(field - for our columns)

- (iii) Key constraint - All the values of primary key must be unique, value of it must not be NULL

- (iv) Entity integrity constraint - No attribute of primary key must contain a null value in any relation

- (v) Referential integrity constraint - All the values taken by foreign key must be either available in relation of primary key or NULL

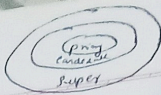
### \* Closure of an Attribute Set -

Set of all those attributes that can be functionally determined from an attribute set is called closure of an attribute set

### Keys in DBMS

A set of attributes that can identify each tuple uniquely in given relation.

Super Key - It is a set of attributes that can identify each tuple uniquely in given relation. It may consist of any no. of attributes



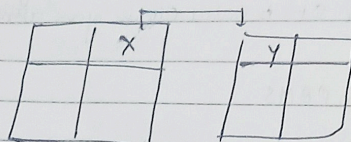
(ii) Candidate key - set of minimal attributes that can identify each tuple uniquely in given relation  
 ↳ represents any attribute

(iii) Primary key - it is a candidate key that a database designer selects while designing a database  
 ↳ Unique  
 ↳ NOT NULL

(iv) Alternate key - candidate keys that are left unimplemented or unused after implementing primary key

(v) Foreign key - an attribute 'X' is called a foreign key to some other attribute 'Y' when its values are dependent on the values of 'Y'

relation where 'Y' is present → referenced relation  
 relation where 'X' is present → referencing relation



Composite key - primary key composed of multiple attributes and not just a single attribute

Unique key - Unique for all the records of the table. Once assigned its value cannot be changed

↳ non-updatable, may have NULL value

### \* Functional dependency -

In Any relation, a functional dependency  $\alpha \rightarrow \beta$  holds if two tuples have same value of attribute  $\alpha$  also have same value for attribute  $\beta$ .

↓  
Types

#### (i) Trivial functional dependency -

A functional dependency  $X \rightarrow Y$  is said to be trivial if and only if  $Y \subseteq X$ .

Thus if RHS of a functional dependency is a subset of LHS

↳ It is called trivial functional dependency

#### (ii) Non-trivial functional dependency -

A functional dependency  $X \rightarrow Y$  is said to be non-trivial if and only if  $Y \not\subseteq X$ .

### + decomposition of a relation -

process of breaking up or dividing a single relation into two or more sub relations

↳ properties of decomposition

(i) Lossy decomposition - It ensures that no information is lost from

original relation during decomposition, when subrelations are joined back  $\rightarrow$  some relation is obtained which was lost decomposed.

- (ii) dependency-preservation - It ensures that none of the functional dependencies that hold the relation are lost
- $\hookrightarrow$  subrelation still satisfy functional dependencies

$\Rightarrow$  Types of decomposition:-

- (i) Lossless join decomposition - If a relation  $R$  is decomposed to subrelations like  $R_1, R_2, R_3, R_4, \dots$  etc.

These subrelations are joined by natural join operator ( $\bowtie$ ) leading to the same relation  $R$

$$R_1 \bowtie R_2 \bowtie R_3 \bowtie R_4 = R$$

- (ii) Lossy join decomposition - If a relation  $R$  is decomposed to subrelations like  $R_1, R_2, R_3, R_4, \dots$  etc.

After joining these subrelations by natural join operation doesn't provide the original relation  $R$  but another relation  $R_n$

$\hookrightarrow$  (lossy join decomposition)

$$R_1 \bowtie R_2 \bowtie R_3 \bowtie R_4 = R_n \supset R$$

## \* Normalization in DBMS-

It is a process of making database consistent by-

- reducing the redundancy
- ensuring the integrity of data by lossless decomposition

## \* Normal forms :-

(i) 1NF - First normal form for a particular relation if each cell in the table has only atomic value

↓  
Attribute of every type is either single valued or Null valued

(ii) 2NF - Second normal form for attribute and only if -

- relation already exists in 1NF form
- No partial dependency

↓  
( $A \rightarrow B$ ) is called partial dependency if and only if  $\rightarrow A$  is a subset of some candidate key and  $B$  is non prime attribute

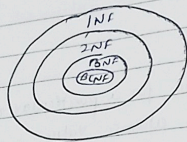
(iii) 3NF - Third normal form for a particular relation if and only if -

- relation already exists for 2NF form
- No transitive dependency.

( $A \rightarrow B$ ) is called transitive dependency if  $A$  is not a super key and  $B$  is non prime attribute

(iii) BCNF (Boyce-Codd Normal form) - A relation is called BCNF for a table if -

- relation already exists in 3NF
- for each non-trivial functional dependency  $A \rightarrow B$ , A is super key of the relation

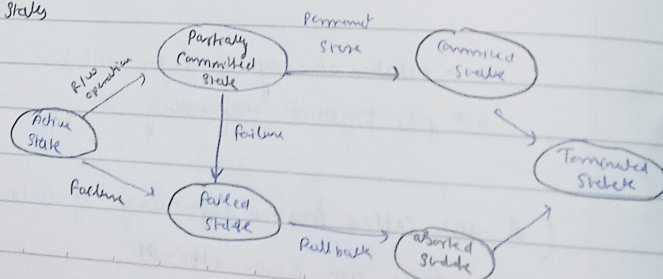


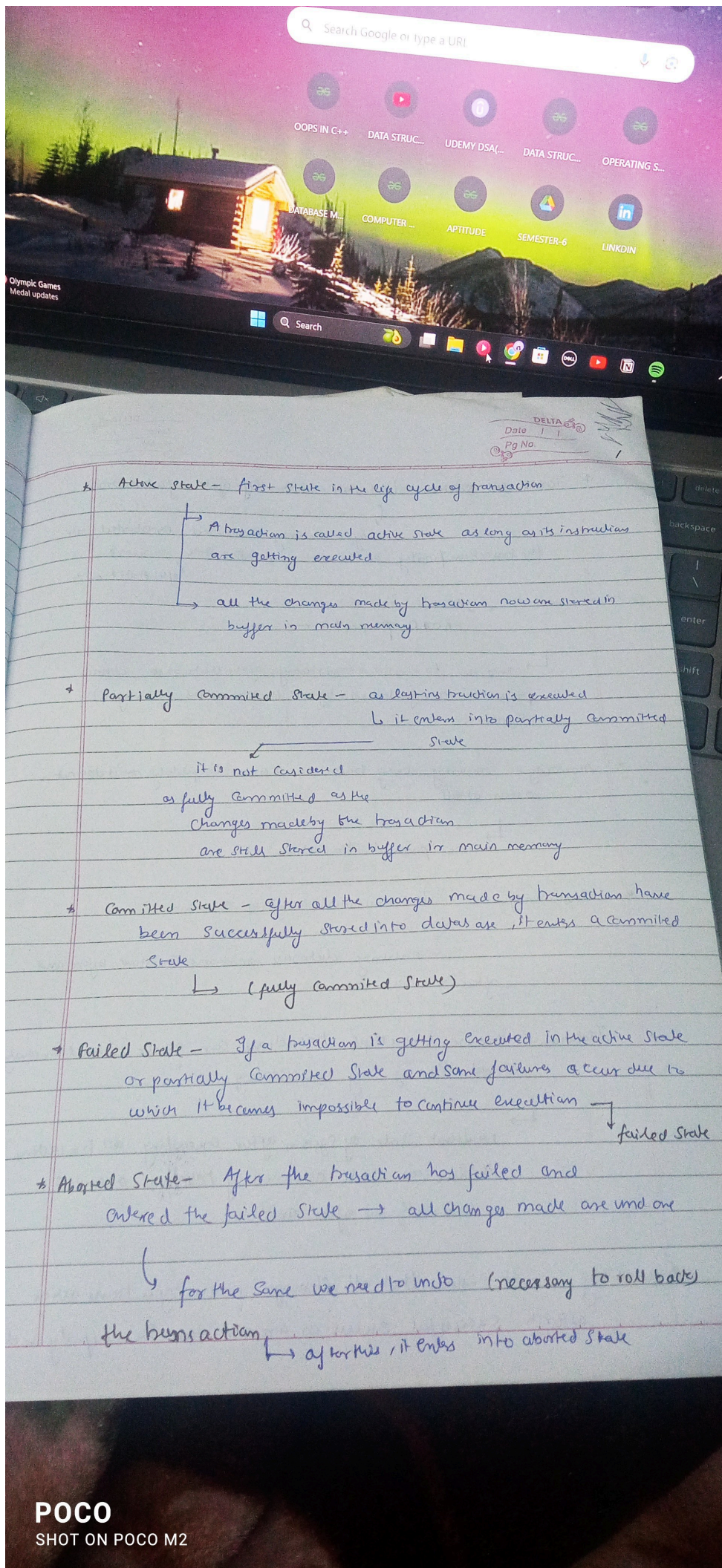
Transaction - A single logical unit of work formed by a set of operations

Operations in transactions -

- Read operation - Read(n) - It will read the value of 'n' from the database and will store it in the buffer in main memory
- Write operation - Write(n) - will write the updated value of 'n' from the buffer to the database

Transaction States





\* Active State - first state in the life cycle of transaction

- A transaction is called active state as long as its instructions are getting executed
- all the changes made by transaction now are stored in buffer in main memory

\* Partially Committed State - as last time transaction is executed  
↳ it enters into partially committed state  
it is not considered as fully committed as the changes made by the transaction are still stored in buffer in main memory

\* Committed State - after all the changes made by transaction have been successfully stored into database, it enters a committed state  
↳ (fully committed state)

\* Failed State - If a transaction is getting executed in the active state or partially committed state and some failures occur due to which it becomes impossible to continue execution  
↓ failed state

\* Aborted State - After the transaction has failed and entered the failed state → all changes made are undone  
↳ for the same we need to undo (necessary to roll back) the transaction  
↳ afterwards, it enters into aborted state

DELTA  
Date / /  
Pg No.

\* Terminated State - Last state in the lifecycle of a transaction  
↳ after entering committed or aborted state, the transaction finally enters into terminated state after that it ends

-ACID properties -

↳ in order to ensure consistency of the database, certain properties are followed by all the transactions occurring in the system

\* Atomicity - ensuring every transaction occurs completely or it does not occur at all

↳ no partial transaction

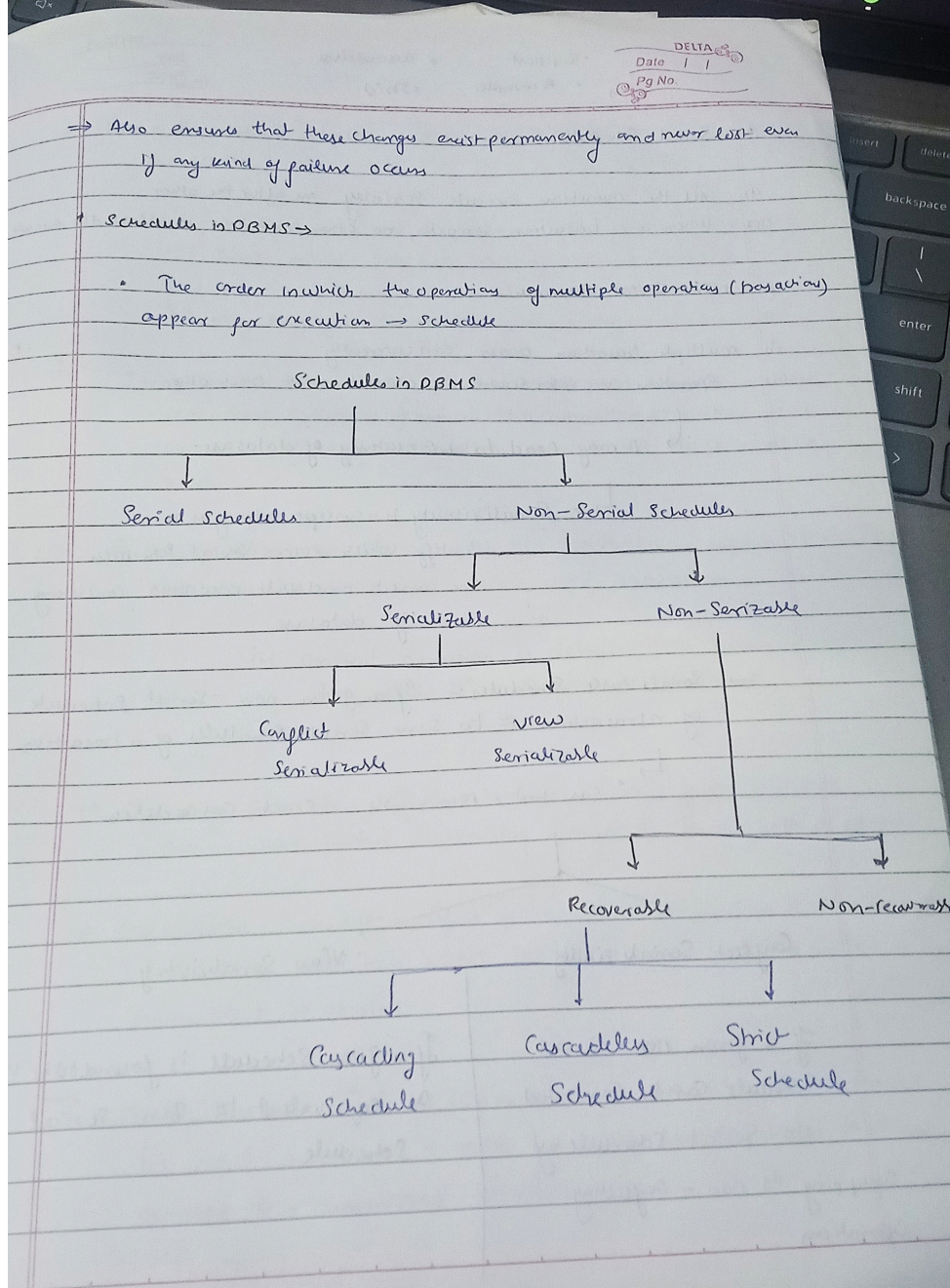
\* Consistency - To maintain integrity constraints

↳ ensuring database remains consistent before and after the transaction

\* Isolation - Ensures multiple transactions can occur simultaneously without causing any inconsistency

↳ resultant state of system after executing all transactions is same as state achieved if transactions executed serially one after another

\* Durability - It ensures that all changes made by a transaction after its successful execution are written successfully on disk



- Consistent
- Recoverable
- Strict
- Cascadeless

### Serial Schedules -

- (i) all the transactions execute serially one after the other
- (ii) When one transaction executes, no other transaction is allowed to execute

### Non-Serial Schedules -

- (i) multiple transactions occur concurrently
- (ii) operations are interleaved or mixed with each other

↳ It may lead to inconsistency of database

↳ • Serializability is a concept helping us to identify which non-serial schedules are correct and will maintain consistency of database

→ Serializable Schedules:- If a given non-serial schedule of transactions is equivalent to some serial schedule of a transaction,

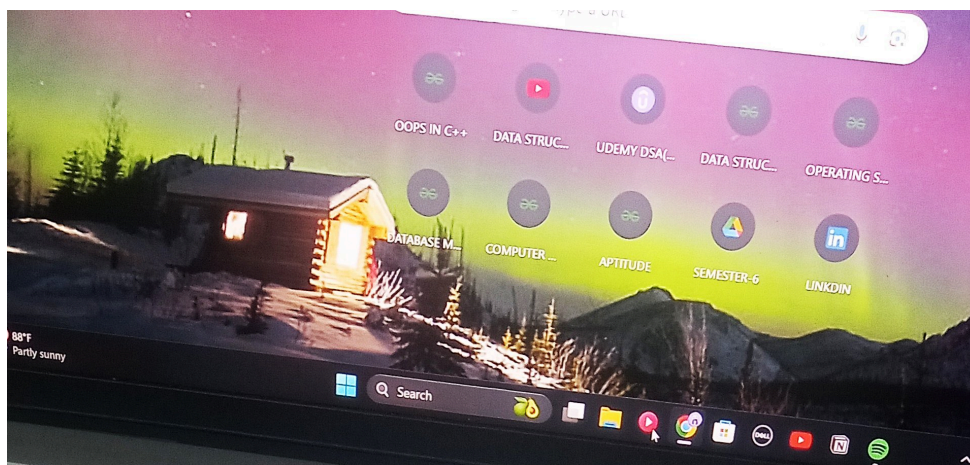
- ↳ • consistent, recoverable, strict, cascadeless

### Conflict Serializability

If a given non-serial schedule can be converted into serial schedule by swapping its non-conflicting operations

### View Serializability

If a given schedule is found to be viewed as equivalent to some serial schedule



→ Non-Serializable Schedule - Non-Serial Schedule is not serializable

↳ it is not guaranteed to produce same effect as produced by some serial schedule on any consistent database

↳ may or may not be consistent, may or may not be recoverable

(i) Irrecoverable Schedules - If in a schedule, a transaction performs dirty read operation from an uncommitted transaction and commits before the transaction from which it has to read the values

(ii) Recoverable Schedules - If in a schedule, a transaction performs dirty read operation from an uncommitted transaction

↓  
its commit operation is delayed till uncommitted transaction either commits or rolls back

⇒ Cascading Schedule - If in a schedule, failure of one transaction causes several other depending transactions to rollback or abort

⇒ Cascadeless Schedule - If in a schedule, a transaction is not allowed to read a data item until last transaction that has written it is committed or aborted

⇒ Strict Schedule - If in a schedule, a transaction is neither allowed to read nor write a data item until last transaction that has written it is committed or aborted

\* Relational algebra - A procedural query language taking a relation as input and generates a relation as output

→ Some operators:-

(i)  $\sigma$  (Selection) - select rows based on given condition

(ii)  $\pi$  (Projection) - project some columns

(iii)  $\times$  → cross product → returns  $(m+n)$  rows where  
 $m$  and  $n$  → rows for  $R_1$  and  $R_2$

(iv)  $\cup$  → Union (tuples that are in  $R_1$  but not  $R_2$ )  
worst case  $R_1 \cup R_2 = m+n$

(v)  $\rho$  (Rename) - renaming a relation to another relation

(vi)  $\cap$  (Intersection) - return common tuples in  $R_1$  and  $R_2$   
↳  $\min(m, n)$  ↳ worst case = 0

(vii)  $\bowtie_c$  → Conditional join (Selection of 2 or more tables, join them based on some condition)

\* B-Trees

↳ every level we have a key and data pointer

↓  
points to either block or record

Root of tree  $\rightarrow$  Can have children b/w 2 and P

$\hookrightarrow$  order of tree

maximum no. of children a node can have

Internal nodes  $\rightarrow$  Children b/w  $[P/2]$  and P  
 $\hookrightarrow$  keys b/w  $[P/2]-1$  to  $(P-1)$

DBMS over File System

$\hookrightarrow$

- data redundancy is less
- data consistency is less
- easy data access
- Security is better, as well as backup, recovery options